

JaxMaterials: A JAX-based package for efficient differentiable material modelling

Yang Chen^{1*} and Eike Hermann Mueller^{1*}

¹ University of Bath, BA2 7EX Claverton Down, United Kingdom * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- Review [↗](#)
- Repository [↗](#)
- Archive [↗](#)

Editor: [↗](#)

Submitted: 26 August 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Many applications in Scientific Computing require not only the fast solution of a forward problem $\theta \rightarrow u \rightarrow J$, which relates some input parameters θ to the solution $u = u(\theta)$ of a partial differential equation (PDE) and ultimately an objective function $J = J(u(\theta))$, but also the computation of sensitivities $\delta J / \delta \theta$. Recently, powerful frameworks such as JAX ([Bradbury et al., 2018](#)) have become available to address this challenge while allowing the user to express the problem at a high abstraction level. We introduce a differentiable JAX-based package for solving an important system of coupled PDEs that arises in continuum mechanics. This allows the efficient modelling of problems which are described by the stationary Cauchy equation together with a user-defined constitutive law that relates microscopic stress σ and strain ε . The user interacts with the library by defining a custom function `compute_sigma(epsilon,param)` which describes this constitutive law that can depend on arbitrary parameters θ encoded in `params`. The code is inherently differentiable and uses the adjoint state method ([Hinze et al., 2008](#)) to propagate gradients through the iterative Lippmann Schwinger solver introduced in ([Moulinec & Suquet, 1998](#)); Anderson acceleration ([Wicht et al., 2021](#)) to improve computational efficiency is also supported. We demonstrate the application of our code to three practical problems that are relevant to the engineering community.

Statement of need

Many materials encountered in engineering have a heterogeneous microstructure. Practically relevant examples include particle-/fibre- reinforced composites, polycrystalline metals, porous solids, and architecture metamaterials. Understanding how microscopic material arrangements determine macroscopic mechanical properties remains a central challenge in materials science and engineering. Numerical simulations have become indispensable for establishing these microstructure-property relationships and for guiding the design of advanced materials.

The mechanical response of heterogeneous materials is commonly described by the equilibrium equation

$$\nabla \cdot \sigma = 0$$

where the local stress field $\sigma(x)$ is linked to the local strain $\varepsilon(x)$ through a constitutive law

$$\sigma = \Sigma(\varepsilon|\theta)$$

with material parameters θ . For example, in linear elasticity problems θ corresponds to the spatially varying elasticity tensor $C(x)$ and the constitutive law reduces to $\sigma_{ij}(x) = \sum_{\ell} C_{ijkl}(x) \varepsilon_{kl}(x)$.

35 Accurate simulation of heterogeneous materials is computationally demanding due to the large
36 separation of spatial scales. The resolution of fine microstructural features requires discretisation
37 on fine computational grids, while strong material contrast, anisotropy and nonlinear constitutive
38 behaviour can make the numerical solution challenging since the discretised problems become
39 ill-conditioned.

40 Forward simulation alone is increasingly insufficient. Material parameters are notoriously difficult
41 to measure directly and must be inferred from experimental data. In addition, physics-based
42 simulations are now frequently combined with machine-learning models in applications such as
43 inverse modelling (Wang et al., 2025) and uncertainty quantification (Akhare et al., 2024), see
44 e.g. (Pestourie et al., 2023). These workflows require efficient computation of sensitivities
45 with respect to model parameters, making differentiable PDE solvers an essential component
46 (Shen et al., 2023).

47 State of the field

48 Since the seminal work of (Moulinec & Suquet, 1998), FFT-based Lippmann-Schwinger
49 solvers have become a standard approach for computing microscopic stress and strain fields in
50 heterogeneous materials. The approach is particularly attractive for simulations on regular grids
51 and on modern parallel hardware (Y. Chen, Gélébart, et al., 2019). Lippmann-Schwinger solvers
52 form the basis of mature software packages such as AMITEX (Gelebart et al., 2020), which
53 provides highly optimised CPU implementations for large-scale material simulations. More
54 recently, GPU-accelerated computing has emerged as an important direction for accelerating
55 large-scale simulation of materials with fine microstructures (Blühdorn et al., 2022; Dutta et
56 al., 2025).

57 Advances in differentiable programming have sparked growing interest in differentiable materials
58 simulations. Frameworks such as JAX (Bradbury et al., 2018) and PyTorch (Paszke et al.,
59 2019) provide automatic differentiation capabilities together with execution on CPUs and GPUs,
60 enabling the integration of PDE solvers into optimisation and machine-learning workflows.
61 In the work of (Pundir & Kammer, 2025), a JAX-based framework is proposed. Users can
62 specify constitutive relations and derivatives are generated automatically by JAX. Efficiency is
63 achieved through just-in-time (JIT) compilation. In a related study, the authors of (Pundir et
64 al., 2026) use automatic differentiation to derive governing equations from energy functionals
65 and solve them using the finite element method. Similarly, (Blühdorn et al., 2022) present a
66 C++ framework that combines automatic differentiation with GPU acceleration.

67 Despite these advances, existing differentiable implementations face important limitations.
68 Directly applying JAX's automatic differentiation to iterative solvers is often computationally
69 unfeasible: Forward-mode differentiation based on Jacobian-vector products (called JVPs in
70 JAX) becomes expensive when the number of input parameters greatly exceeds the number of
71 objective quantities, as is common in real applications. Reverse-mode differentiation is usually
72 more efficient, but backpropagating through iterative solvers presents two challenges. First, all
73 intermediate states must be stored during the forward pass, resulting in substantial memory
74 costs. Second, iterative solvers rely on dynamic stopping criteria, whereas efficient reverse-
75 mode differentiation in JAX requires knowledge of the number of iterations at (just-in-time)
76 compile time. The adjoint-state method addresses these challenges, and it has become widely
77 adopted in finite element frameworks, for example through the pyadjoint library (Mitusch et
78 al., 2019), which is integrated with Firedrake (Farrell et al., 2013; Rathgeber et al., 2016).
79 The resulting toolchain has recently been applied to materials modelling (Farsi et al., 2025).
80 However, most differentiable PDE frameworks are built on finite element discretisations which,
81 although highly flexible, are generally less computationally efficient than FFT-based methods
82 for microstructure simulations on structured grids. Existing differentiable FFT implementations,
83 such as (Pundir & Kammer, 2025), leverage automatic differentiation but are not primarily
84 designed for memory-efficient reverse-mode differentiation of iterative solvers.

Main achievements

Our work combines the computational efficiency of FFT-based Lippmann-Schwinger solvers (Moulinec & Suquet, 1998; Schneider, 2021) with the flexibility of differentiable programming in JAX. Users can define arbitrary constitutive laws $\sigma = \Sigma(\varepsilon|\theta)$ through a simple Python interface while benefiting from JIT compilation and GPU acceleration. The adjoint-state method allows memory-efficient reverse-mode differentiation and support for dynamical stopping criteria in the iterative solver. To our best knowledge, this is the first open-source framework that combines a modular constitutive-law interface, efficient adjoint-based differentiation, and FFT-based computational homogenisation within a unified JAX ecosystem. The resulting software can be used as a key component in topology optimisation, inverse parameter identification, uncertainty quantification and machine-learning-assisted materials modelling.

Mathematical background

To motivate the design of the software we briefly review the relevant mathematical details.

PDE Problem

We consider the following system of partial differential equations for spatially varying strain ε and stress σ :

$$\begin{aligned} \sum_j \partial_j \sigma_{ij} &= 0 && \text{(Cauchy momentum equation)} \\ \sigma_{ij} &= \Sigma_{ij}(\varepsilon|\theta) \quad \text{with } \varepsilon_{kl} = \varepsilon_{kl}^* + \bar{\varepsilon}_{kl} && \text{(Constitutive law)} \quad (1) \\ \varepsilon_{kl}^* &= \frac{1}{2} (\partial_k u_l + \partial_l u_k) && \text{(Strain-displacement relation)} \end{aligned}$$

The problem is solved in a cuboid domain with periodic boundary conditions for the displacement field $u(x)$ and for given average strain $\bar{\varepsilon}$.

The problem-dependent constitutive law $\sigma = \Sigma(\varepsilon|\theta)$ depends on the parameters θ . Special cases are:

- General linear materials with $\sigma_{ij}(x) = \sum_{k\ell} C_{ijkl}(x) \varepsilon_{k\ell}(x)$ where $C(x) =: \theta$ is the spatially varying elasticity tensor
- Isotropic linear materials for which $C_{ijkl}(x) = \lambda(x) \delta_{ij} \delta_{kl} + \mu(x) (\delta_{ik} \delta_{jl} + \delta_{il} \delta_{jk})$. In this case θ encapsulates the two scalar-valued Lamé parameters $\{\mu(x), \lambda(x)\} =: \theta$.

Lippmann Schwinger iteration

The problem in (Equation 1) can be written in the form

$$\varepsilon + \Gamma^0 * (\Sigma(\varepsilon|\theta) - C^0 \varepsilon) = \bar{\varepsilon}. \quad (2)$$

In this equation $*$ denotes convolution and C^0 is the homogeneous isotropic elasticity tensor described by the two reference Lamé parameters $\mu^0, \lambda^0 \in \mathbb{R}$. The operator Γ^0 is constructed from the tensor-valued Green's function of the corresponding PDE (see appendix of (Moulinec & Suquet, 1998)).

As discussed in (Moulinec & Suquet, 1998), the self-consistent equation in (Equation 2) is solved by exploiting the fact that Γ^0 is diagonal in Fourier space and by applying the Lippmann Schwinger iteration

$$\varepsilon^{(s+1)} = \bar{\varepsilon} - \mathcal{F}^{-1} \circ \hat{\Gamma}^0 \circ \mathcal{F} \tau^{(s)} \quad \text{with } \tau^{(s)} = (\Sigma(\varepsilon^{(s)}|\theta) - C^0 \varepsilon^{(s)}) \text{ and } \varepsilon^{(0)} = \bar{\varepsilon}$$

Here $\mathcal{F}$ and $\mathcal{F}^{-1}$ denote the forward and inverse Fourier transform respectively.

To improve robustness we adopted the rotated discretisation scheme of (Willot, 2015) which is also used in AMITEX (Gelebart et al., 2020). As in (Y. Chen, Gélébart, et al., 2019), Anderson acceleration (Wicht et al., 2021) is used to reduce the number of iterations. For linear stress-strain relations, this is equivalent to a preconditioned GMRES iteration (Walker & Ni, 2011), but it can also be used for non-linear constitutive laws.

Reverse-mode differentiation with the adjoint method

Let $J = J(\varepsilon, \sigma)$ be the objective function representing, for example, the average stress, total strain energy, or effective bulk modulus (see below for examples). We want to compute the sensitivities of J

$$\frac{\delta J}{\delta \theta}, \quad \frac{\delta J}{\delta \bar{\varepsilon}}$$

subject to the condition that strain $\varepsilon = \varepsilon(\theta, \bar{\varepsilon})$ and stress $\sigma = \sigma(\theta, \bar{\varepsilon})$ satisfy the equations in (Equation 1) for given $\bar{\varepsilon}$ and material parameters θ .

Since every step of the Lippmann Schwinger iteration consists of elementary, differential operations, in principle the sensitivities can be obtained with JAX automatic differentiation capabilities provided the constitutive law $\sigma = \Sigma(\varepsilon|\theta)$ is differentiable. However, reverse mode-differentiation is not available for while-loops since the number of iterations is unknown at (just-in-time) compile time. To address this, we employ the adjoint state method (Hinze et al., 2008; Johnson, 2012). This leads to an adjoint Lippmann Schwinger equation of a very similar structure as (Equation 2):

$$\Lambda + (\Gamma^0 * \Lambda) \frac{\delta \Sigma}{\delta \varepsilon} - \lambda^0 \text{tr}(\Gamma^0 * \Lambda) \mathbb{I} - 2\mu^0 (\Gamma^0 * \Lambda) = - \left(\frac{\delta J}{\delta \varepsilon} + \frac{\delta J}{\delta \sigma} \frac{\delta \Sigma}{\delta \varepsilon} \right). \quad (3)$$

The adjoint problem in (Equation 3) is solved iteratively with Anderson acceleration. From the adjoint state Λ the derivatives of the objective function J with respect to the parameters θ and the average strain $\bar{\varepsilon}$ can be computed as

$$\frac{\delta J}{\delta \theta} = \left(\frac{\delta J}{\delta \sigma} + \Gamma^0 * \Lambda \right) : \frac{\delta \Sigma}{\delta \theta}, \quad \frac{\delta J}{\delta \bar{\varepsilon}} = - \int_{\Omega} \Lambda(z) dz.$$

Software design

The overarching design principle of the code is to realise a separation of concerns between the user-defined physical model encoded in the constitutive law $\Sigma(\varepsilon|\theta)$ and the algorithms which solve the resulting PDE system. The approach can be interpreted as an example of Dependency Inversion (Martin, 1996): the generic solver algorithm gets passed a function which implements an implicitly defined interface for $\Sigma(\varepsilon|\theta)$. This approach allows the application of the code to a wide range of engineering problems. Differentiability is achieved by building on the powerful JAX library, which allows the composition of the solver with other algorithms. This is demonstrated in the examples below, where we show how JaxMaterials can be used as an ingredient of sophisticated workflows in material modelling.

150 JAX implementation

151 Since the code is based on JAX, all functions are pure and parameters are passed as state
152 variables. The central functionality is exposed through the function `lippmann_schwinger()`
153 which gets passed as a user-defined constitutive law $\Sigma(\varepsilon|\theta)$ of the form:

```
def compute_sigma(epsilon, params):
    # Compute stress sigma from strain epsilon, given params
    return sigma
```

154 Internally, `lippmann_schwinger()` calls a backend function which is equipped with custom
155 reverse mode gradients through JAX's `defvjp` functionality. It should be stressed that
156 `compute_sigma()` can be any function, as long as it is reverse mode differentiable. By
157 design our library allows the implementation of non-trivial models such as the one in (Y. Chen,
158 Vasiukov, et al., 2019). Other examples are given below.

159 For convenience, special cases for isotropic and anisotropic elastic materials have been
160 implemented as well. In this case the user only needs to pass the Lamé coefficients $\lambda(x)$, $\mu(x)$
161 or the independent entries of the (symmetric) elasticity tensor $C(x)$.

162 Example usage

163 Consider an isotropic elastic material for which $\sigma_{ij}(x) = \lambda(x) \text{tr}(\varepsilon(x)) \delta_{ij} + 2\mu(x) \varepsilon_{ij}(x)$. In
164 this case $\theta = \{\mu(x), \lambda(x)\}$ and the constitutive law $\Sigma(\varepsilon|\theta)$ is implemented as the following
165 function:

```
def compute_sigma(epsilon, params):
    tr_epsilon = epsilon[0, ...] + epsilon[1, ...] + epsilon[2, ...]
    sigma = 2 * params["mu"] * epsilon + params["lambda"] * jnp.stack(
        3 * [tr_epsilon] + 3 * [jnp.zeros(epsilon.shape[-3:], dtype=epsilon.dtype)]
    )
    return sigma
```

166 As usual, we followed (Voigt, 1928) to represent the 3×3 symmetric matrix ε with components
167 ε_{ij} as a vector of the form $(\varepsilon_{00}, \varepsilon_{11}, \varepsilon_{22}, \varepsilon_{01}, \varepsilon_{02}, \varepsilon_{12})^T$. The parameters are passed as a
168 dictionary which constitutes a JAX pytree:

```
params = {"lambda": lambda, "mu": mu}
```

169 In the code, we first import the necessary libraries

```
import numpy as np
import jax
```

```
from jax import numpy as jnp
```

```
from jaxmaterials.common import GridSpec
from jaxmaterials.solver.lippmann_schwinger import lippmann_schwinger
```

170 and construct a structured grid of the domain $\Omega = [0, 1] \times [0, 1] \times [0, \frac{1}{2}]$ with $32 \times 32 \times 16$
171 voxels:

```
nx, ny, nz = 32, 32, 16
grid_spec = GridSpec(nx, ny, nz, Lx=1.0, Ly=1.0, Lz=0.5)
```

172 In this example, the Lamé parameters $\mu(x)$, $\lambda(x)$ are set to random values and only the first
173 component of the mean strain $\bar{\varepsilon}$ is nonzero:

```
rng = np.random.default_rng(seed=47273)
mu = rng.uniform(low=0.8, high=1.1, size=(nx, ny, nz)).astype(np.float32)
lambda = rng.uniform(low=0.6, high=0.7, size=(nx, ny, nz)).astype(np.float32)
```

```
epsilon_bar = np.array([1, 0, 0, 0, 0, 0], dtype=np.float32)
params = {"lambda": lambda, "mu": mu}
```

174 As in (Moulinec & Suquet, 1998) the reference Lamé parameters are obtained by
175 averaging the largest and smallest values across the domain $\mu^0 = [\mu]$, $\lambda^0 = [\lambda]$ with
176 $[f] := \frac{1}{2} (\min_{x \in \Omega} \{f(x)\} + \max_{x \in \Omega} \{f(x)\})$:

```
ref_params = {
    key: (np.min(value) + np.max(value)) / 2 for (key, value) in params.items()
}
```

177 Assume that we are interested in the objective function $J = \varepsilon^2 + \sigma^2$. This can be implemented
178 as a function of $\theta, \bar{\varepsilon}$ by calling the differentiable Lippmann Schwinger solver inside the function
179 which implements J :

```
def loss_fn(params, epsilon_bar):
    epsilon, sigma = lippmann_schwinger(
        compute_sigma, params, epsilon_bar, ref_params, grid_spec
    )
    return jnp.sum(epsilon**2 + sigma**2)
```

180 Finally, gradients of J with respect to $\theta, \bar{\varepsilon}$ can be computed by using JAX's autodiff capabilities,
181 for example

```
grad_fn = jax.grad(loss_fn, argnums=(0, 1))
g_params, g_epsilon_bar = grad_fn(params, epsilon_bar)
```

182 Special case: isotropic elastic material

183 In this particular case we can also use the bespoke function `lippmann_schwinger_isotropic()`
184 which assumes an isotropic elastic constitutive law. This function only needs to be passed the
185 Lamé parameters $\lambda(x), \mu(x)$ as `params = {"lambda": lambda, "mu": mu}` and automatically
186 computes μ^0, λ^0 :

```
from jaxmaterials.solver.lippmann_schwinger import lippmann_schwinger_isotropic

def loss_fn(params, epsilon_bar):
    epsilon, sigma = lippmann_schwinger_isotropic(
        params, epsilon_bar, grid_spec=grid_spec
    )
    return jnp.sum(epsilon**2 + sigma**2)
```

187 Low-level CUDA-C implementation

188 In addition to the Python code, a low level CUDA-C implementation for elastic materials is also
189 provided and it can be accessed through the same interface functions. Note that this needs to
190 be compiled separately (with CMake). Depending on the hardware, this can be faster than
191 the JAX code, in particular for the isotropic case. At the moment, the CUDA implementation
192 is not differentiable, but in principle this limitation could be overcome by implementing the
193 adjoint Lippmann Schwinger solve in CUDA.

194 If we are only interested in the forward solve for the isotropic material discussed above, we
195 can use the efficient CUDA implementation by simply passing the `use_cuda=True` keyword to
196 isotropic Lippmann-Schwinger solver:

```
epsilon, sigma = lippmann_schwinger_isotropic(
    params, epsilon_bar, grid_spec, use_cuda=True
)
```

197 Research impact statement

198 Three selected applications demonstrate how the differentiable FFT solver in JaxMaterials can
199 be integrated into materials research workflows.

200 Topology optimisation

201 We used JaxMaterials to design periodic porous metamaterials that maximise the effective bulk
202 modulus K at prescribed solid volume fractions. The optimality criteria method (Bendsøe et al.,
203 2004) requires the computation of the gradient $\delta J / \delta \rho(x)$ of the objective function $J = -K$
204 with respect to the spatially varying density ρ in each step of the outer optimisation loop. The
205 effective bulk modulus K is computed with the energy-based method in (Z. Chen et al., 2022)
206 which requires solving (Equation 1) subject to a macroscopic strain load $\bar{\varepsilon} = (1, 1, 1, 0, 0, 0)^T$
207 to obtain the microscopic stress $\sigma(x)$ which is averaged to the macroscopic stress $\bar{\sigma}$:

$$K = \frac{1}{9} \sum_{i,j=1}^3 C_{ijij}^{(\text{eff})} = \frac{1}{9} \bar{\varepsilon}^T : \bar{\sigma} \quad \text{with} \quad \bar{\sigma} = \frac{1}{|\Omega|} \int_{\Omega} \sigma(x) dx \quad (4)$$

208 One realisation of the material is characterised by a tuple consisting of the density $\rho(x) \in$
209 $[0, 1]$ and a dictionary mat of real-valued numbers which include the Poisson ratio ν and
210 the parameters E_0, E_1, ρ_0, p of the SIMP model which parametrises Young's modulus
211 $E(x) = E_0 + (E_1 - E_0)(\rho(x) + \rho_0)^p$ as a function of $\rho(x)$. The Lamé parameters $\lambda(x),$
212 $\mu(x)$ are then obtained from $E(x), \nu$ in the following helper function:

```
def lame_coefficients(rho, mat):
    E = mat["E0"] + (mat["E1"] - mat["E0"]) * (rho + mat["kk"]) ** mat["penalty"]
    lmbda = E * mat["nu"] / (1.0 + mat["nu"]) / (1.0 - 2.0 * mat["nu"])
    mu = E / (2.0 * (1.0 + mat["nu"]))
    return lmbda, mu
```

213 This allows the implementation stress-strain relationship as the user-defined function
214 `compute_sigma_from_density()`, which maps the local strain $\varepsilon(x)$ and all material
215 parameters (collected in `params`) to the local stress $\sigma(x)$ by assuming linear isotropic
216 behaviour:

```
def compute_sigma_from_density(epsilon, params):
    lmbda, mu = lame_coefficients(*params)
    tr_epsilon = epsilon[0, ...] + epsilon[1, ...] + epsilon[2, ...]
    sigma = 2 * mu * epsilon + lmbda * jnp.stack(
        3 * [tr_epsilon] + 3 * [jnp.zeros(epsilon.shape[-3:], dtype=epsilon.dtype)]
    )
    return sigma
```

217 With this, the objective function J can be computed for a given $\rho(x)$ by solving (Equation 1)
218 for σ and averaging over the domain to obtain the macroscopic stress $\bar{\sigma}$ which is used in the
219 computation of K in (Equation 4).

```
def objective_fn(rho, mat, grid_spec):
    lmbda, mu = lame_coefficients(rho, mat)
    params = {"lambda": lmbda, "mu": mu}
    ref_params = {
        key: jax.lax.stop_gradient(0.5 * (jnp.max(value) + jnp.min(value)))
        for key, value in params.items()
    }
    epsilon_bar = jnp.array([1.0, 1.0, 1.0, 0.0, 0.0, 0.0])
    epsilon, sigma = lippmann_schwinger(
        compute_sigma_from_density, (rho, mat), epsilon_bar, ...
```

```
)
sigma_bar = jnp.mean(sigma, axis=[1, 2, 3])
bulk_modulus = jnp.sum(
    epsilon_bar[:3] * sigma_bar[:3] + epsilon_bar[3:] * sigma_bar[3:] * 2
)
return -bulk_modulus / 9
```

Because `lippmann_schwinger()` provides a custom reverse-mode derivative based on the adjoint-state method, the function `objective_fn()` is reverse mode differentiable with respect to the density $\rho(x)$. The gradient $\delta J / \delta \rho$ can be computed directly using the standard JAX interface:

```
value_grad_fn = jax.value_and_grad(objective_fn, argnums=0, has_aux=False)
J, dJ = value_grad_fn(rho, mat, grid_spec)
```

Results

Numerical experiments were carried out for three different solid volume fractions $\phi = |\Omega|^{-1} \int_{\Omega} \rho(x) dx$. Each design is represented by a cubic representative volume element (RVE) of size $0.5 \times 0.5 \times 0.5 \text{ mm}^3$. The solid phase has Young's modulus of $E_1 = 1 \text{ GPa}$ and a Poisson's ratio of $\nu = 0.3$, while the void phase is approximated by a much softer material with Young's modulus of $E_0 = 10^{-6} \text{ GPa}$. The combination of high porosity, complex pore morphology, and a stiffness contrast of $E_0 / E_1 = 10^6$ makes the solution of the fundamental equations in (Equation 1) numerically challenging. Anderson acceleration effectively addresses this challenge as demonstrated in (Y. Chen, Gélébart, et al., 2019).

Figure 1 shows the evolution of the topologies described by $\rho(x)$, where each step of the outer optimisation requires the computation of the gradients $\delta J / \delta \rho(x)$ as described above.

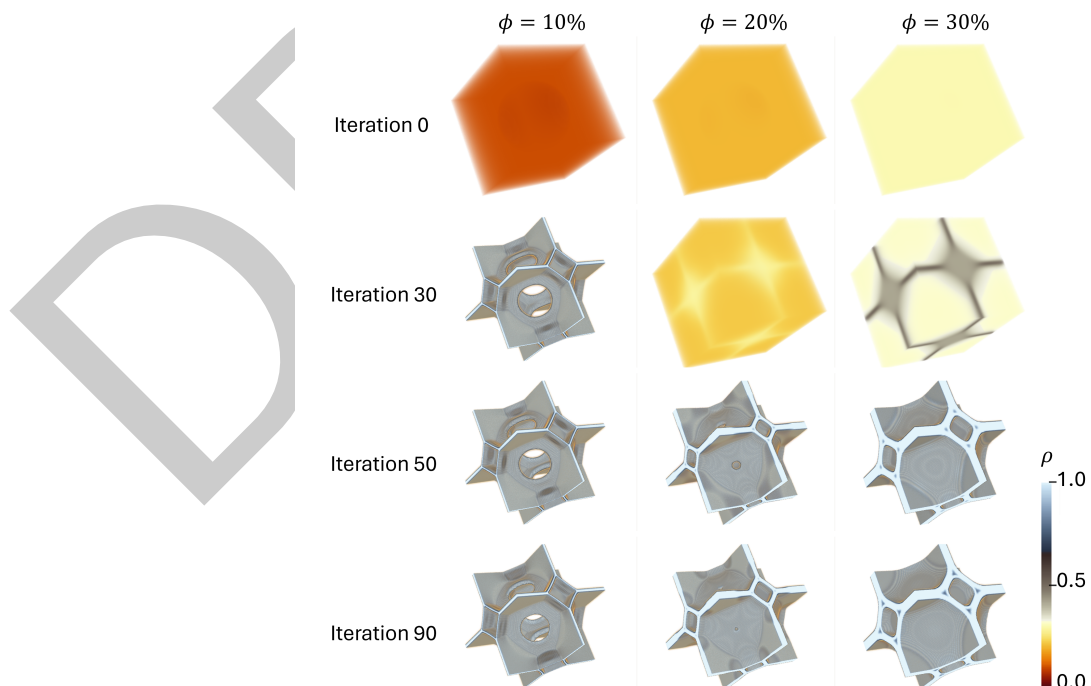


Figure 1: Evolution of the optimised porous structures at solid volume fractions of 10% (left), 20% (middle), and 30% (right).

The corresponding convergence histories in Figure 2 confirm that for all three volume fractions

236 ϕ the effective bulk modulus converges towards a stable plateau. This demonstrates that
237 JaxMaterials' forward and adjoint solutions provide sufficiently stable sensitivities for gradient-
238 based topology optimisation.

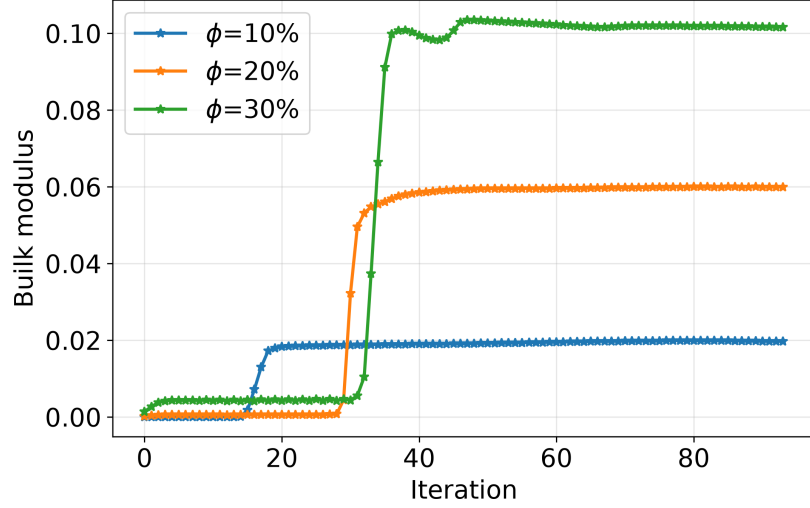


Figure 2: Evolution of the effective bulk modulus during topology optimisation at different solid volume fractions.

239 All three simulations were completed within approximately one hour on an Nvidia RTX A6000
240 GPU.

241 Phase-field fracture problem

242 To demonstrate that JaxMaterials is readily embedded in a non-trivial multiphysics workflow,
243 we consider the variational phase-field fracture model of (Miehe et al., 2010). This setup
244 couples a nonlinear mechanical equilibrium problem of the form (Equation 1) to the evolution
245 of a scalar, spatially varying damage field $d(x) \in [0, 1]$. Here $d = 0$ denotes intact material
246 and $d = 1$ corresponds to fully damaged material.

247 The phase field subproblem is

$$\frac{g_c}{l_c} [d - l_c^2 \Delta d] = 2(1 - d) \mathcal{H}(\varepsilon) \quad (5)$$

248 where g_c is the critical energy release rate, l_c controls the regularised fracture length scale,
249 and $\mathcal{H}$ is the strain-dependent history field that stores the maximum tensile elastic energy
250 attained at each material point and enforces irreversible damage evolution:

$$\mathcal{H}(x, t) := \max_{\tau \in [0, t]} \psi^+(\varepsilon(x, \tau)). \quad (6)$$

251 By projecting onto the positive and negative eigenmodes, the strain tensor $\varepsilon = \varepsilon_+ + \varepsilon_-$ is
252 split into a tensile mode ε_+ and a compressive part ε_- .

253 Since fractures do not grow under compression only the tensile component of the strain is
254 sensitive to damage and the stress-strain relationship can be modelled as

$$\sigma = ((1 - d)^2 + k_{\text{stab}}) [\lambda \langle \text{tr}(\varepsilon) \rangle_+ \mathbb{I} + 2\mu \varepsilon_+] + [\lambda \langle \text{tr}(\varepsilon) \rangle_- \mathbb{I} + 2\mu \varepsilon_-] \quad (7)$$

with $z_{\pm} = \frac{1}{2}(z \pm |z|)$ for $z \in \mathbb{R}$. The small stabilisation parameter $k_{\text{stab}} \ll 1$ prevents σ from becoming degenerate as $d \rightarrow 1$. Observe that while the constitutive law is of the form $\Sigma(\varepsilon|\theta)$ required in (Equation 1), the relationship between stress and strain is no longer linear and significantly more complicated than in the previous examples: the separation of the strain into tensile and compressive components requires an eigenvalue decomposition which is a highly non-linear operation.

Following (Y. Chen, Vasiukov, et al., 2019), the coupled mechanical equations (Equation 1) with $\Sigma(\varepsilon|\theta)$ defined by (Equation 7) and the phase-field equation in (Equation 5) are solved sequentially. For this, a staggered scheme alternates between the following three steps to obtain time-dependent strain $\varepsilon(x, t_n)$, history $\mathcal{H}(x, t_n)$ and phase field $d(x, t_n)$:

- **Step 1:** Given the history field $\mathcal{H}(x, t_n)$, solve (Equation 5) with the FFT algorithm described in (Y. Chen, Vasiukov, et al., 2019) to obtain the updated phase field $d(x, t_{n+1})$.
- **Step 2:** Given $d(x, t_{n+1})$, use JaxMaterials to solve (Equation 1) and obtain the updated strain field $\varepsilon(x, t_{n+1})$.
- **Step 3:** Given $\varepsilon(x, t_{n+1})$, compute the updated history field $\mathcal{H}(x, t_{n+1})$ according to (Equation 6).

In the computation of $\varepsilon(x, t_{n+1})$ in Step 2 the constitutive law $\Sigma(\varepsilon|\theta)$ depends on the following three parameters:

1. the phase field d
2. the constant Lamé parameters $\mu, \lambda \in \mathbb{R}$ in (Equation 7)
3. the stabilisation parameter k_{stab}

The constitutive law is described by the following user-defined function, which gets passed the parameters $\theta := [\lambda, \mu, d, k_{\text{stab}}]$ through the variable params:

```
def compute_sigma_damaged(epsilon, params):
    lmbda, mu, d, k_stab = params
    eps_tensor = voigt_to_tensor(epsilon)
    tr_eps = jnp.trace(eps_tensor, axis1=-2, axis2=-1)
    sigma = {}
    for sign, op in (( "+", jnp.maximum), ( "-", jnp.minimum)):
        tr_eps_signed = op(tr_eps, 0.0)
        eigvals, eigvecs = jnp.linalg.eigh(eps_tensor)
        eigvals_signed = op(eigvals, 0.0)
        eps_signed_tensor = jnp.einsum(
            "...ia,...a,...ja->...ij", eigvecs, eigvals_signed, eigvecs
        )
        eps_signed_v = tensor_to_voigt(eps_signed_tensor)
        sigma[sign] = 2.0 * mu * eps_signed_v
        sigma[sign] = sigma[sign].at[:3].add(lmbda * tr_eps_signed)
    return ((1.0 - d[None, ...]) ** 2 + k_stab) * sigma["+"] + sigma["-"]
```

The small helper functions `tensor_to_voigt()` and `voigt_to_tensor()` convert between the 3×3 representation of a symmetric matrix A with components A_{ij} and its vector-valued Voigt representation $(A_{00}, A_{11}, A_{22}, A_{01}, A_{02}, A_{12})^T$ (Voigt, 1928). Since the function `compute_sigma_damaged()` is implemented entirely using differentiable JAX operations, it can be passed to JaxMaterials:

```
epsilon, sigma = lippmann_schwinger(compute_sigma_damaged, (lmbda, mu, d, k_stab), ...)
```

Results

Figure 3 shows the simulated fracture pattern in a particle-reinforced composite. The complete staggered simulation took 2.6 hours on an NVIDIA RTX A6000 GPU.

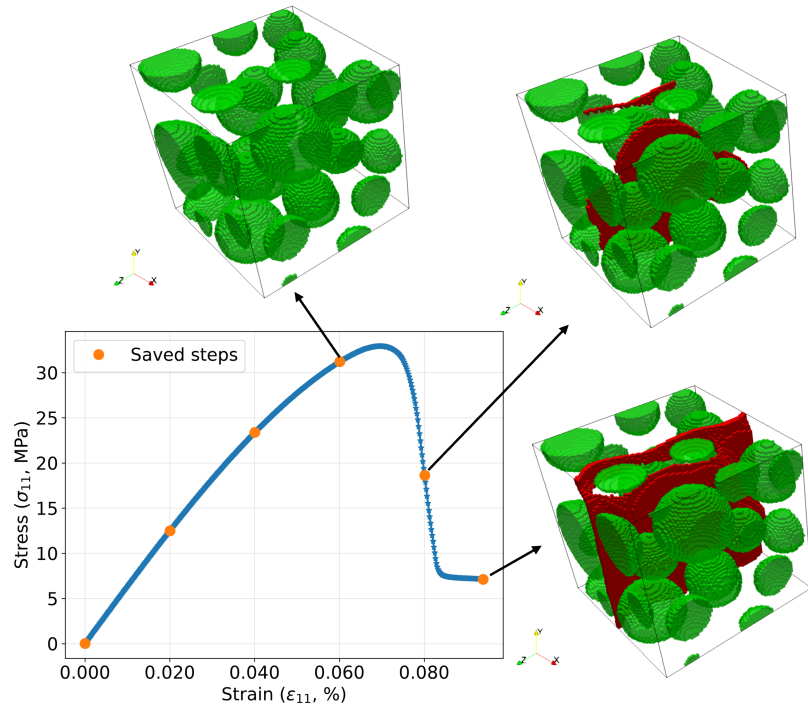


Figure 3: Phase-field fracture simulation of a particle-reinforced composite. JaxMaterials solves the nonlinear mechanical equilibrium problem at each staggered iteration.

Inverse problem: material parameter identification

The final example demonstrates the use of JaxMaterials for gradient-based identification of constituent material properties from macroscopic measurements. In a common scenario, the location of particles in a reinforced composite can be readily inferred from X-ray tomography, but the material properties of the particle and matrix are unknown and need to be inferred from a measured macroscopic stress-strain relationship. More specifically, we assume that for a given macroscopic load $\bar{\varepsilon}$ the corresponding stress $\bar{\sigma} = \bar{\sigma}(\bar{\varepsilon}, u)$ depends on four parameters collected in the vector u which contains the Young modulus E and Poisson ratio ν of the particles and matrix respectively:

$$u = (E^{\text{particle}}, E^{\text{matrix}}, \nu^{\text{particle}}, \nu^{\text{matrix}}) \in \mathbb{R}^4.$$

The goal of the inverse problem is then to infer the material parameters u from a measured stress $\bar{\sigma}^{\text{exp}}$ for prescribed strain $\bar{\varepsilon}^{\text{exp}}$, assuming that the microscopic stress σ solves the material equations in (Equation 1) with constitutive law $\sigma = \Sigma(\varepsilon|u)$. For this, the residual

$$r(u) = \frac{1}{|\Omega|} \int_{\Omega} \sigma(\bar{\varepsilon}^{\text{exp}}, u) dx - \bar{\sigma}^{\text{exp}}$$

is minimised with the Newton-Raphson method (in the code the residuals for several different load vectors $\bar{\varepsilon}^{\text{exp}}$ are combined into a longer vector). Starting from some initial guess u_0 , the parameters are updated according to $u_{k+1} = u_k + \Delta u_k$ where Δu_k is obtained by solving the overdetermined system

$$J(u_k) \Delta u_k = -r(u_k), \quad J(u_k) = \frac{\delta r}{\delta u} \Big|_{u=u_k} \quad (8)$$

303 with the method of normal equations.

304 To compute the Jacobian matrix $J = J(u)$ for a given parameter vector u , JaxMaterials
305 solves the heterogeneous elasticity problem for the prescribed strain state $\bar{\epsilon}^{\text{exp}}$ and parameters
306 u . The macroscopic stress $\bar{\sigma}^{\text{sim}}$ is obtained by averaging the local stress $\sigma(\bar{\epsilon}^{\text{exp}}, u)$. Since
307 all these operations are differentiable, the residual $r = r(u)$ and its Jacobian J are readily
308 obtained in JAX. For this, define a differentiable function `forward_solve()` which gets passed
309 the material properties u and average strain $\bar{\epsilon}^{\text{exp}}$. This function constructs the spatially varying
310 Lamé parameters $\lambda(x)$, $\mu(x)$ from the topology and the material parameters in u and calls
311 `lippmann_schwinger_isotropic()` to compute the stress σ , which is averaged and returned.
312 The following JAX code then computes the Jacobian, which is subsequently used in the outer
313 Newton-Raphson iteration given in (Equation 8):

```
residual_fn = lambda u: forward_solve(u, epsilon_bar_exp, ...) - sigma_exp
jacobian_fn = jax.jacobian(residual_fn)
r = residual_fn(u)
J = jacobian_fn(u)
```

314 This implementation separates the outer optimisation algorithm from the inner mechanics
315 solver. No problem-specific sensitivity equations need to be derived or implemented by the
316 user.

317 Results

318 Synthetic experimental data was generated by simulations for fixed topology and given material
319 parameters u^{exp} , using JaxMaterials as a fast forward model. A datapoint consists of a
320 prescribed macroscopic strain $\bar{\epsilon}^{\text{exp}}$ and corresponding macroscopic stress $\bar{\sigma}^{\text{exp}} = \bar{\sigma}(\bar{\epsilon}^{\text{exp}}, u^{\text{exp}})$.
321 The output u of the above Newton-Raphson solver can then be compared to the ground truth
322 u^{exp} .

323 The workflow is summarised in Figure 4.

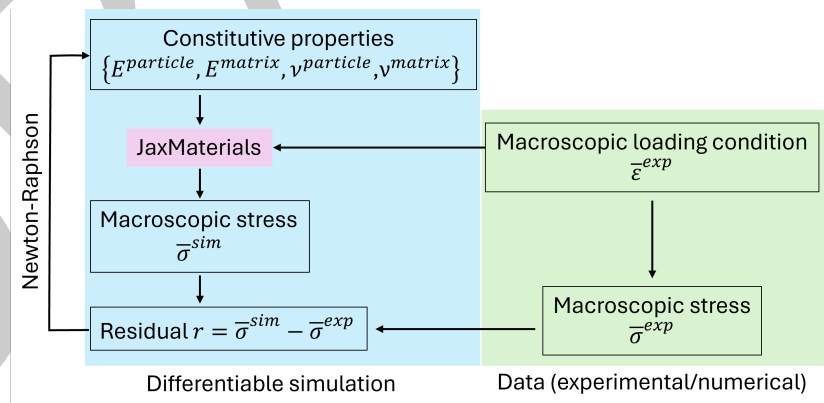


Figure 4: JaxMaterials-based workflow for identifying constituent properties from macroscopic stress-strain measurements.

324 Figure 5 shows the convergence history for the material parameters E^{particle} , E^{matrix} , ν^{particle} , ν^{matrix} .
325 All four parameters converged to their target values within four Newton iterations. This
326 demonstrates that the sensitivities supplied by JaxMaterials can support efficient parameter
327 identification from macroscopic data.

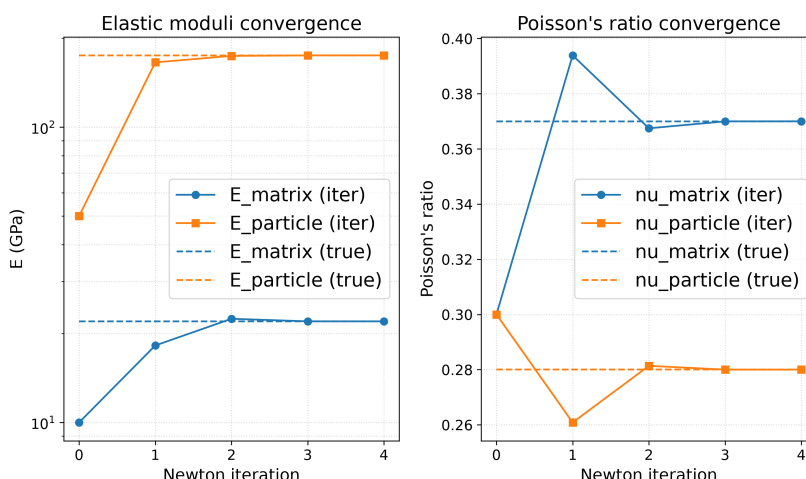


Figure 5: Convergence of the identified particle and matrix properties during the Newton-Raphson iterations.

AI usage disclosure

Generative AI (GitHub copilot and ChatGPT) was used to generate some of the code and automated workflow scripts, to identify and locate bugs and to generate informed feedback on the code and documentation. Code generation was closely supervised by limiting it to routine transformations and by breaking it into small, transparent tasks the correctness implementation of which could be easily verified. All generated code and all AI suggestions were carefully reviewed by the authors. GitHub copilot was used for final spell checking of the manuscript.

Outlook and future work

There are several ways in which this work can be extended.

First of all, the code could be parallelised across multiple GPUs to enable large scale simulations. Further extending the CUDA code to also support reverse mode differentiation can improve performance. However, since JAX just-in-time compilation produces highly efficient code, it is less clear whether the required effort will justify this. So far, we have only considered the staggered discretisation also used in (Gelebart et al., 2020) and employed periodic boundary conditions. It would in principle also be possible to extend the code to other discretisations and boundary conditions (see (Gelebart, 2024)). Here we concentrated on the tensor-valued equations which are solved for microscopic stress and strain; the solution of these equations is often the computational bottleneck of simulations. However, the same approach could be applied to scalar valued equations of the form $\nabla \cdot E = f$ with $E = E(\phi|\theta)$. For example, the constitutive law $E = -K\nabla\phi$ would describe a heat flux problem with thermal conductivity K . We welcome suggestions for other possible extensions from users with specific applications.

The examples presented in this work use JaxMaterials for classical workflows in mechanical engineering and material design. Another very important application is to embed the differentiable solver in Scientific Machine Learning approaches. For example, in (Pestourie et al., 2023) the authors solve a low-fidelity surrogate PDE, where the input parameters are learned from the high-fidelity input fields through a neural network. Our code can be naturally embedded in larger JAX-based ML workflows to implement the approach in (Pestourie et al., 2023) for problems in material modelling.

Acknowledgements

We thank Xuanchen Liu for contributing to the initial setup of the phase-field fracture example during his final-year project.

References

- Akhare, D., Chen, Z., Gulotty, R., Luo, T., & Wang, J.-X. (2024). Probabilistic physics-integrated neural differentiable modeling for isothermal chemical vapor infiltration process. *Npj Computational Materials*, 10(1), 120.
- Bendsøe, M. P., Sigmund, O., & others. (2004). *Topology optimization: Theory, methods, and applications* (Vol. 2). Springer Berlin.
- Blühdorn, J., Gauger, N. R., & Kabel, M. (2022). AutoMat: Automatic differentiation for generalized standard materials on GPUs. *Computational Mechanics*, 69(2), 589–613.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Katariya, Y., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). <http://github.com/jax-ml/jax>
- Chen, Y., Gélébart, L., Chateau, C., Bornert, M., Sauder, C., & King, A. (2019). Analysis of the damage initiation in a SiC/SiC composite tube from a direct comparison between large-scale numerical simulation and synchrotron x-ray micro-computed tomography. *International Journal of Solids and Structures*, 161, 111–126.
- Chen, Y., Vasiukov, D., Gélébart, L., & Park, C. H. (2019). A FFT solver for variational phase-field modeling of brittle fracture. *Computer Methods in Applied Mechanics and Engineering*, 349, 167–190.
- Chen, Z., Wu, B., Xie, Y. M., Wu, X., & Zhou, S. (2022). FFT-based inverse homogenization for cellular material design. *International Journal of Mechanical Sciences*, 231, 107572.
- Dutta, T., Mohan, D., Shenoy, S., Attar, N., Kalokhe, A., Sagar, A., Bhure, S., Pradhan, S. S., Praharaj, J., Mridha, S., & others. (2025). MicroSim: A high-performance phase-field solver based on CPU and GPU implementations. *Computational Materials Science*, 246, 113438.
- Farrell, P. E., Ham, D. A., Funke, S. W., & Rognes, M. E. (2013). Automated derivation of the adjoint of high-level transient finite element programs. *SIAM Journal on Scientific Computing*, 35(4), C369–C393.
- Farsi, A., Bouziani, N., & Ham, D. A. (2025). Missing physics discovery through fully differentiable finite element-based machine learning. *arXiv Preprint arXiv:2507.15787*.
- Gelebart, L. (2024). FFT-based simulations of heterogeneous conducting materials with combined non-uniform Neumann, periodic and Dirichlet boundary conditions. *European Journal of Mechanics-A/Solids*, 105, 105248.
- Gelebart, L., Derouillat, J., Doucet, N., Ouaki, F., Marano, A., & Duverge, J. (2020). *AMITEX code*. <https://amitexfft.github.io/AMITEX/>.
- Hinze, M., Pinnau, R., Ulbrich, M., & Ulbrich, S. (2008). *Optimization with PDE constraints*. Springer Science & Business Media.
- Johnson, S. G. (2012). Notes on adjoint methods for 18.335. *Introduction to Numerical Methods*, 732.
- Martin, R. C. (1996). The dependency inversion principle. *C++ Report*, 8(6), 61–66.

- 399 Miehe, C., Hofacker, M., & Welschinger, F. (2010). A phase field model for rate-independent
400 crack propagation: Robust algorithmic implementation based on operator splits. *Computer*
401 *Methods in Applied Mechanics and Engineering*, 199(45-48), 2765–2778.
- 402 Mitusch, S., Funke, S., & Dokken, J. (2019). Dolfin-adjoint 2018.1: Automated adjoints for
403 FEniCS and firedrake. *Journal of Open Source Software*, 4(38), 1292.
- 404 Moulinec, H., & Suquet, P. (1998). A numerical method for computing the overall response
405 of nonlinear composites with complex microstructure. *Computer Methods in Applied*
406 *Mechanics and Engineering*, 157(1-2), 69–94.
- 407 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin,
408 Z., Gimelshein, N., Antiga, L., & others. (2019). Pytorch: An imperative style, high-
409 performance deep learning library. *Advances in Neural Information Processing Systems*,
410 32.
- 411 Pestourie, R., Mroueh, Y., Rackauckas, C., Das, P., & Johnson, S. G. (2023). Physics-
412 enhanced deep surrogates for partial differential equations. *Nature Machine Intelligence*,
413 5(12), 1458–1465.
- 414 Pundir, M., & Kammer, D. S. (2025). Simplifying FFT-based methods for solid mechanics
415 with automatic differentiation. *Computer Methods in Applied Mechanics and Engineering*,
416 435, 117572.
- 417 Pundir, M., Lorez, F., & Kammer, D. S. (2026). A versatile FEM framework with native
418 GPU scalability via globally-applied AD. *Computer Methods in Applied Mechanics and*
419 *Engineering*, 460, 119118.
- 420 Rathgeber, F., Ham, D. A., Mitchell, L., Lange, M., Luporini, F., McRae, A. T., Bercea, G.-T.,
421 Markall, G. R., & Kelly, P. H. (2016). Firedrake: Automating the finite element method by
422 composing abstractions. *ACM Transactions on Mathematical Software (TOMS)*, 43(3),
423 1–27.
- 424 Schneider, M. (2021). A review of nonlinear FFT-based computational homogenization
425 methods: M. schneider. *Acta Mechanica*, 232(6), 2051–2100.
- 426 Shen, C., Appling, A. P., Gentine, P., Bandai, T., Gupta, H., Tartakovsky, A., Baity-Jesi, M.,
427 Fenicia, F., Kifer, D., Li, L., & others. (2023). Differentiable modelling to unify machine
428 learning and physical models for geosciences. *Nature Reviews Earth & Environment*, 4(8),
429 552–567.
- 430 Voigt, W. (1928). Lehrbuch der kristallphysik (textbook of crystal physics). *BG Teubner*,
431 *Leipzig Und Berlin*.
- 432 Walker, H. F., & Ni, P. (2011). Anderson acceleration for fixed-point iterations. *SIAM Journal*
433 *on Numerical Analysis*, 49(4), 1715–1735.
- 434 Wang, X., Yin, Z.-Y., Wu, W., & Zhu, H.-H. (2025). Differentiable finite element method
435 with galerkin discretization for fast and accurate inverse analysis of multidimensional
436 heterogeneous engineering structures. *Computer Methods in Applied Mechanics and*
437 *Engineering*, 437, 117755.
- 438 Wicht, D., Schneider, M., & Böhlke, T. (2021). Anderson-accelerated polarization schemes
439 for fast fourier transform-based computational homogenization. *International Journal for*
440 *Numerical Methods in Engineering*, 122(9), 2287–2311.
- 441 Willot, F. (2015). Fourier-based schemes for computing the mechanical response of composites
442 with accurate local fields. *Comptes Rendus Mécanique*, 343(3), 232–245.